

Relazione tecnica

Python pt.2 - Esercizio principale e facoltativo

D'Angelo Giovanni

1. Introduzione

Nel corso dell'esercitazione ho realizzato due programmi in Python. Nel primo ho sviluppato una funzione capace di ricevere in ingresso una lista di parole e di restituire una nuova lista contenente la lunghezza di ciascun elemento. Nel secondo, previsto come facoltativo, ho creato un generatore di password in grado di produrre una password semplice oppure complessa in base alla scelta dell'utente.

L'attività è stata svolta in ambiente Kali Linux, utilizzando terminale ed editor di testo per la scrittura, il salvataggio e il collaudo dei file Python. Ho eseguito entrambi i programmi direttamente dal terminale, verificando il funzionamento sia con il comando `python` sia con `python3` quando necessario.

2. Svolgimento dell'esercizio principale

2.1 Preparazione del file

Per prima cosa ho creato la cartella di lavoro dedicata agli esercizi Python e mi sono spostato al suo interno. Successivamente ho creato il file `esercizio_parole.py` e l'ho aperto nell'editor per inserire il codice richiesto.

2.2 Struttura del programma

All'interno del file ho definito una prima funzione chiamata `char_counter(lista_a)`. Questa funzione inizializza una lista vuota, scorre una per una le parole presenti nella lista ricevuta in input e, tramite `len(parola)`, inserisce nella nuova lista il numero di caratteri corrispondente a ciascuna parola.

Ho poi aggiunto una seconda funzione, `char_counter_pro(lista_a)`, che svolge lo stesso compito in forma più compatta, utilizzando una list comprehension. In questo modo ho riportato entrambe le possibili soluzioni richieste dall'esercizio: quella classica con ciclo `for` e `append`, e quella sintetica in un'unica istruzione.

Per il test ho definito la lista `lista_test = ["ciao", "python", "rete", "socket"]` e ho stampato a schermo il risultato prodotto da entrambe le funzioni, così da controllare immediatamente la correttezza dell'output.

2.3 Esecuzione e verifica

Dopo aver salvato il file ho eseguito il programma dal terminale. Il risultato ottenuto è stato `[4, 6, 4, 6]` per la prima funzione e nuovamente `[4, 6, 4, 6]` per la versione compatta. L'output è corretto perché le parole della lista di prova contengono rispettivamente 4, 6, 4 e 6 caratteri.

Per ulteriore verifica ho avviato lo stesso file anche con `python3`, ottenendo lo stesso comportamento. Questo passaggio mi ha confermato che il codice era stato scritto e salvato correttamente e che la logica del programma rispondeva in modo preciso alla traccia.

```
kali@kali: ~/python_esercizi
Session Actions Edit View Help
(kali@kali)-[~]
$ mkdir -p ~/python_esercizi
cd ~/python_esercizi
touch esercizio_parole.py
nano esercizio_parole.py

(kali@kali)-[~/python_esercizi]
$ python esercizio_parole.py
# Se serve Python 3
python3 esercizio_parole.py
[4, 6, 4, 6]
[4, 6, 4, 6]
[4, 6, 4, 6]
[4, 6, 4, 6]

(kali@kali)-[~/python_esercizi]
$
```

Figura 1 - Esecuzione dell'esercizio principale con stampa corretta delle due liste di lunghezze.

3. Svolgimento dell'esercizio facoltativo

3.1 Creazione del generatore di password

Nel file password_generator.py ho importato i moduli random e string e ho definito la funzione psw_generator(). All'interno della funzione ho predisposto due insiemi di caratteri: il primo composto da cifre, lettere e punteggiatura per la password complessa; il secondo composto solo da cifre e lettere per la password semplice.

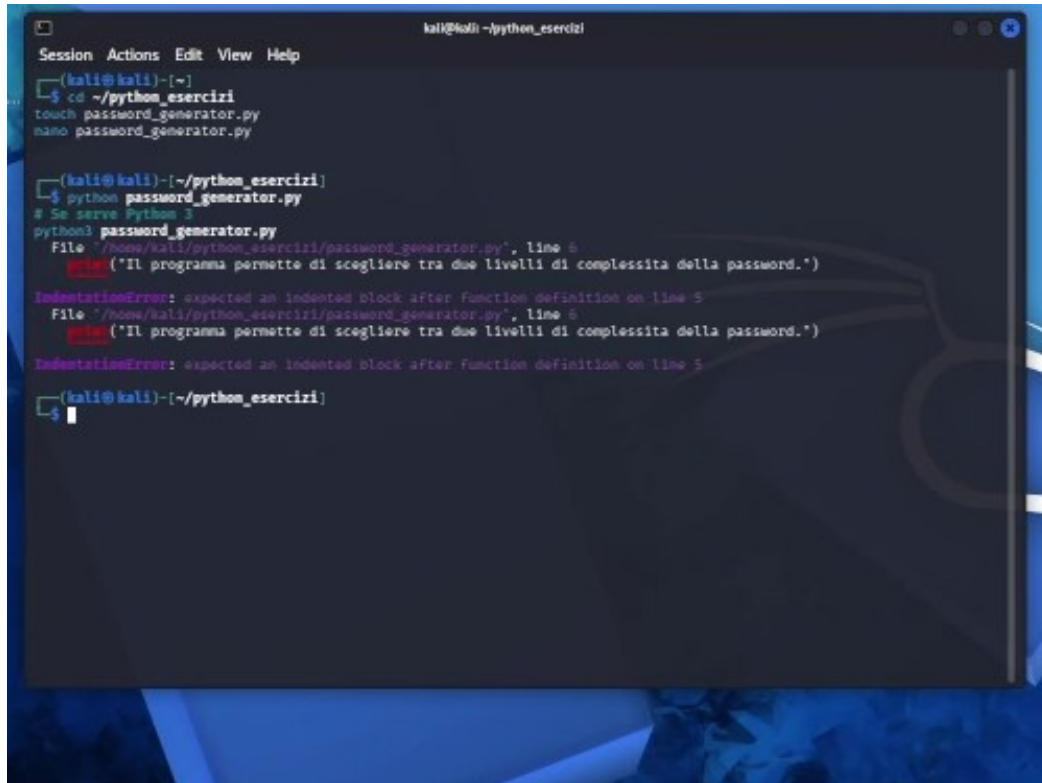
Ho quindi impostato una scelta tramite input, chiedendo all'utente se desiderasse una password semplice oppure complessa. Nel caso della scelta C ho assegnato lunghezza 20 e insieme ascii_chars; in caso diverso ho assegnato lunghezza 8 e insieme alphanum_chars. Infine ho costruito la password con un ciclo while, concatenando ogni volta un carattere selezionato casualmente fino al raggiungimento della lunghezza prevista.

3.2 Errore riscontrato durante il primo avvio

Durante il primo tentativo di esecuzione il programma non è partito correttamente. Il terminale ha infatti restituito un errore di indentazione subito dopo la definizione della funzione. In un primo momento ho controllato anche altri dettagli del testo, ma il problema reale non era legato ai contenuti della stringa: era dovuto alla formattazione del codice.

La causa concreta era una piccola distrazione nel copia e incolla dei passaggi preparati in precedenza: alcune istruzioni si erano saldate in modo scorretto oppure erano andate a capo in una posizione sbagliata,

alterando l'indentazione che Python richiede dopo la riga `def psw_generator()`. Questo mi ha confermato in modo molto chiaro che, in Python, anche una sola parola o una sola riga fuori posto può bloccare completamente l'esecuzione del programma.

A screenshot of a terminal window titled 'kali@kali: ~/python_esercizi'. The terminal shows the following commands and output:

```
(kali@kali)-[~]  
$ cd ~/python_esercizi  
touch password_generator.py  
nano password_generator.py  
  
(kali@kali)-[~/python_esercizi]  
$ python password_generator.py  
# Se serve Python 3  
python3 password_generator.py  
File ~/home/kali/python_esercizi/password_generator.py, line 4  
SyntaxError: ("Il programma permette di scegliere tra due livelli di complessità della password.")  
  
IndentationError: expected an indented block after function definition on line 5  
File ~/home/kali/python_esercizi/password_generator.py, line 6  
SyntaxError: ("Il programma permette di scegliere tra due livelli di complessità della password.")  
  
IndentationError: expected an indented block after function definition on line 5  
  
(kali@kali)-[~/python_esercizi]  
$
```

Figura 2 - Errore di indentazione comparso durante il primo tentativo di esecuzione del generatore di password.

3.3 Correzione del file e risoluzione del problema

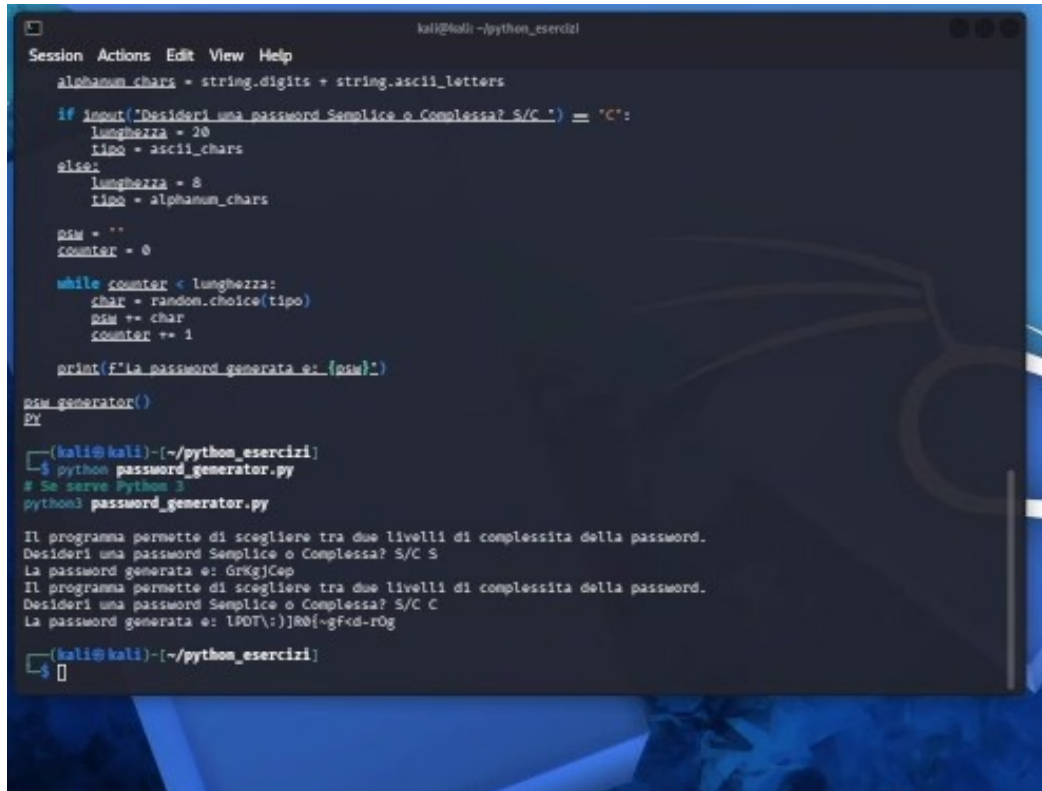
Per risolvere il problema ho riaperto il file, ho controllato il posizionamento delle righe una per una e ho rimesso ogni istruzione nel blocco corretto. In particolare ho ripristinato l'indentazione interna della funzione, separando correttamente le righe che si erano fuse durante il trasferimento del testo e ristabilendo la struttura logica del programma.

Una volta sistemato il file ho salvato nuovamente il contenuto e ho ripetuto l'esecuzione dal terminale. A quel punto il programma ha chiesto correttamente se generare una password semplice o complessa e ha portato a termine la creazione della password senza ulteriori errori.

3.4 Verifica finale del funzionamento

Nel collaudo finale ho verificato entrambe le modalità. Inserendo S il programma ha generato una password semplice da 8 caratteri alfanumerici; inserendo C ha prodotto una password complessa da 20 caratteri ASCII. In entrambi i casi l'output è stato coerente con quanto previsto dalla traccia e il programma si è concluso correttamente.

Questo secondo esercizio è stato utile non solo per completare il compito, ma anche per ribadire un aspetto pratico molto importante: il codice può essere logicamente corretto, ma se l'organizzazione delle righe e l'indentazione non sono perfette Python interrompe l'esecuzione immediatamente.



```
kali@kali: ~/python_esercizi
Session Actions Edit View Help
alphanumeric_chars = string.digits + string.ascii_letters

if input("Desideri una password Semplice o Complessa? S/C ") == "C":
    lunghezza = 20
    tipo = ascii_chars
else:
    lunghezza = 8
    tipo = alphanumeric_chars

psw = ""
counter = 0

while counter < lunghezza:
    char = random.choice(tipo)
    psw += char
    counter += 1

print(f"La password generata e: {psw}")

psw_generator()
PY

(kali@kali)-[~/python_esercizi]
$ python password_generator.py
# Se serve Python 3
python3 password_generator.py

Il programma permette di scegliere tra due livelli di complessita della password.
Desideri una password Semplice o Complessa? S/C S
La password generata e: GrKgjCep
Il programma permette di scegliere tra due livelli di complessita della password.
Desideri una password Semplice o Complessa? S/C C
La password generata e: lPOT\:]@0[-gfd-r0g

(kali@kali)-[~/python_esercizi]
$
```

Figura 3 - Verifica finale del programma con richiesta del tipo di password e generazione corretta dell'output.

4. Conclusioni

L'esercitazione mi ha permesso di applicare in modo pratico due aspetti diversi della programmazione in Python. Con il primo file ho lavorato sulle funzioni e sulla gestione delle liste, mostrando sia una soluzione classica sia una compatta. Con il secondo file ho utilizzato moduli standard, input da tastiera, condizioni, concatenazione di caratteri e cicli.

Il punto più significativo emerso durante il lavoro è stato il controllo dell'errore nel facoltativo. La correzione del file mi ha fatto verificare concretamente quanto sia importante mantenere il codice ordinato e ben indentato, perché anche una disattenzione minima può impedire l'avvio del programma. Una volta sistemata la struttura, l'esecuzione è risultata corretta e completa in entrambe le parti dell'esercizio.